



Rapport de Projet

Recherche opérationnelle – flots dans les graphes

Hugo CLAVEILLE

Master 1 Informatique

7 mai 2026

Table des matières

1	Introduction	3
2	Structure de données	4
2.1	Représentation générale du graphe	4
2.2	Structure d'un arc	4
2.3	Graphe résiduel	5
2.4	Lecture des données	5
3	Flot maximum	6
3.1	Principe	6
3.2	Recherche d'un chemin augmentant	6
3.3	Mise à jour du graphe résiduel	7
3.4	Coupe minimale	7
3.5	Résultat sur l'exemple de la consigne	7
4	Flot de coût minimum	8
4.1	Objectif	8
4.2	Chemins augmentants de coût minimum	8
4.3	Gestion des coûts négatifs	9
4.4	Version avec Dijkstra et renormalisation	9
4.5	Résultat sur l'exemple de la consigne	9
5	Détection de cycles négatifs	11

5.1	Pourquoi c'est nécessaire	11
5.2	Algorithme utilisé	11
5.3	Normalisation	11
6	Visualisation et tests	13
6.1	Export DOT	13
6.2	Tests effectués	13
7	Choix et limites	14
7.1	Choix principaux	14
7.2	Limites actuelles	14
8	Conclusion	16

1. Introduction

Ce projet porte sur des problèmes classiques de flots dans les graphes : le flot maximum, le flot de coût minimum et la détection de cycles négatifs. L'objectif était de construire les algorithmes nous-mêmes, sans utiliser de librairie spécialisée de type NetworkX, afin de mieux comprendre le rôle du graphe résiduel et la manière dont les solutions évoluent au fil des augmentations.

Le dépôt est organisé autour d'une structure de graphe commune, utilisée ensuite par les différents algorithmes. Cette organisation m'a paru importante : au lieu de coder chaque méthode avec son propre format de données, tous les traitements manipulent les mêmes objets `Graph` et `Edge`. Cela rend les tests plus simples, mais surtout cela met en évidence que les algorithmes de flot reposent tous sur la même idée centrale : modifier progressivement les capacités résiduelles des arcs.

Les principaux fichiers du projet sont les suivants :

- `src/graph_struct.py` : définition des structures de données et lecture des graphes depuis les fichiers texte ;
- `src/max_flow.py` : calcul d'un flot maximum avec une variante de Ford-Fulkerson basée sur un parcours en largeur ;
- `src/min_cost_flow.py` : calcul d'un flot de coût minimum par chemins augmentants de plus court coût ;
- `src/Dijkstra_min_cost_flow.py` : seconde version avec la même interface, utilisée dans les tests de comparaison ;
- `src/detect_negative_cycle.py` : détection de cycles négatifs par Bellman-Ford et normalisation simple des coûts ;
- `src/graph_to_dot.py` : export des graphes au format DOT pour visualiser les graphes et les graphes résiduels.

Le rapport présente donc les choix faits dans le code, les structures de données utilisées, puis les algorithmes implémentés et les résultats obtenus sur les exemples de test.

2. Structure de données

2.1 Représentation générale du graphe

Le graphe est représenté par une classe `Graph`. Elle contient :

- `n`, le nombre de sommets ;
- `source`, le sommet source ;
- `sink`, le sommet puits ;
- `adj`, une liste d'adjacence.

Le choix d'une liste d'adjacence est naturel pour ce projet. Les algorithmes parcourent souvent les arcs sortants d'un sommet donné, par exemple lors d'un BFS ou lors des relaxations de Bellman-Ford. Une matrice d'adjacence aurait été plus simple pour chercher un arc précis, mais beaucoup moins agréable pour parcourir uniquement les arcs réellement présents.

Chaque case `adj[u]` contient donc la liste des arcs qui partent du sommet `u`. Cette représentation est aussi assez compacte : elle ne stocke pas les arcs inexistants.

2.2 Structure d'un arc

Les arcs sont représentés par une dataclass `Edge` :

```
@dataclass
class Edge:
    origin: int
    dest: int
    cap: int
    cost: int
    retour: bool = False
```

Un arc contient son origine, sa destination, sa capacité résiduelle, son coût unitaire et un booléen `retour`. Ce dernier sert à distinguer les arcs ajoutés dans le sens inverse lors de la construction du graphe résiduel. Ce choix est surtout utile pour l'affichage DOT :

les arcs de retour peuvent être colorés différemment ou masqués selon le niveau de détail voulu.

2.3 Graphe résiduel

Le graphe résiduel est construit directement dans la méthode `add_edge`. Lorsqu'on ajoute un arc (u, v) de capacité c et de coût w , le programme ajoute aussi un arc inverse (v, u) de capacité initiale 0 et de coût $-w$.

```
fwd = Edge(origin=_origin, dest=_dest, cap=cap, cost=cost)
rev = Edge(origin=_dest, dest=_origin, cap=0, cost=-cost, retour=True)
```

Ce choix est un point central du projet. Le flot envoyé sur un arc direct est traduit par une diminution de sa capacité résiduelle. En parallèle, la capacité de l'arc inverse augmente, ce qui permet à un algorithme futur de revenir partiellement sur une décision prise plus tôt.

Par exemple, si l'on envoie une quantité f sur l'arc (u, v) , le code applique :

$$c_{uv} \leftarrow c_{uv} - f$$

$$c_{vu} \leftarrow c_{vu} + f$$

Cela évite de stocker séparément le flot courant sur chaque arc. Le flot est implicite : il est lisible à partir de l'évolution des capacités résiduelles. C'est plus léger, mais cela demande d'être vigilant au moment d'interpréter les résultats.

2.4 Lecture des données

Les graphes sont chargés depuis des fichiers texte avec le format suivant :

Ligne	Signification
<code>n m s t</code>	nombre de sommets, nombre d'arcs, source, puits
<code>u v cap cost</code>	arc de <code>u</code> vers <code>v</code> , capacité, coût

La fonction `load_graph_from_file` ignore les lignes vides et les commentaires, ce qui permet de garder des fichiers de test lisibles. Les fichiers `test2.txt` et `test3.txt`, dans le dossier `data`, reprennent notamment les exemples donnés dans la consigne.

3. Flot maximum

3.1 Principe

Le flot maximum est traité dans `src/max_flow.py`. Le code suit l'idée de Ford-Fulkerson : tant qu'il existe un chemin augmentant de la source au puits dans le graphe résiduel, on envoie le plus de flot possible sur ce chemin.

Dans cette implémentation, le chemin augmentant est trouvé avec un parcours en largeur. On obtient donc une variante de type Edmonds-Karp. Ce choix rend l'algorithme plus stable qu'une recherche arbitraire de chemin, car le BFS sélectionne un chemin contenant un nombre minimal d'arcs.

3.2 Recherche d'un chemin augmentant

La fonction `bfs` explore uniquement les arcs dont la capacité résiduelle est strictement positive. Pour chaque sommet atteint, elle mémorise son prédécesseur, ce qui permet ensuite de reconstruire le chemin complet de la source vers le puits.

Elle conserve aussi la quantité maximale de flot que l'on peut encore envoyer jusqu'à chaque sommet. Cette quantité correspond au minimum des capacités résiduelles rencontrées sur le chemin.

Si le puits est atteint, la fonction renvoie :

- la liste des arcs du chemin augmentant ;
- la quantité de flot que l'on peut pousser sur ce chemin.

Si aucun chemin n'existe, elle renvoie un flot nul, ce qui arrête l'algorithme principal.

3.3 Mise à jour du graphe résiduel

Une fois un chemin trouvé, la fonction `ford_fulkerson` met à jour les capacités résiduelles :

```
edge.cap -= flow
rev_edge.cap += flow
```

Cette opération est répétée pour tous les arcs du chemin. La variable `max_flow` accumule la quantité totale envoyée.

3.4 Coupe minimale

La consigne demande aussi une coupe minimale. Dans l'approche par graphe résiduel, elle se déduit naturellement après le calcul du flot maximum. Une fois qu'il n'existe plus de chemin augmentant, on peut faire un dernier parcours depuis la source en ne suivant que les arcs de capacité résiduelle positive.

Les sommets encore atteignables forment un ensemble S , et les autres sommets forment T . Les arcs du graphe initial qui partent de S vers T constituent alors une coupe minimale. Le code actuel calcule bien le graphe résiduel final ; la coupe peut donc être retrouvée à partir de cet état, même si elle n'est pas encore affichée par une fonction dédiée.

3.5 Résultat sur l'exemple de la consigne

Le fichier `data/test2.txt` correspond au problème de flot maximum donné dans la consigne. Le programme renvoie :

$$\text{flot maximum} = 60$$

Ce résultat est cohérent avec la capacité sortante de la source :

$$20 + 30 + 10 = 60$$

La source peut donc envoyer au plus 60 unités, et l'algorithme parvient effectivement à envoyer ces 60 unités jusqu'au puits.

4. Flot de coût minimum

4.1 Objectif

Le problème du flot de coût minimum ajoute une contrainte économique au problème de flot. Il ne suffit plus d'envoyer beaucoup de flot : il faut aussi minimiser le coût total.

Dans le code, le coût total est calculé en accumulant, à chaque augmentation :

$$\text{coût total} \leftarrow \text{coût total} + f \times c(P)$$

où f est le flot envoyé sur le chemin et $c(P)$ le coût du chemin choisi.

4.2 Chemins augmentants de coût minimum

La fonction principale du fichier `src/min_cost_flow.py` garde la même structure générale que pour le flot maximum. La différence vient de la fonction de recherche du chemin : au lieu de choisir un chemin court en nombre d'arcs, on cherche un chemin de coût minimum dans le graphe résiduel.

Pour cela, la fonction `shortest_path` utilise Bellman-Ford. Ce choix est pertinent car le graphe résiduel peut contenir des arcs de coût négatif : chaque arc inverse a un coût opposé à celui de l'arc direct. Dijkstra classique ne peut pas être utilisé directement dans ce cas sans transformation des coûts.

L'algorithme effectue donc $n - 1$ phases de relaxation, puis reconstruit le chemin vers le puits grâce au tableau des prédécesseurs. La capacité du chemin est ensuite le minimum des capacités résiduelles sur ses arcs.

4.3 Gestion des coûts négatifs

Les coûts négatifs apparaissent naturellement avec les arcs de retour. Ils ne sont pas un problème en eux-mêmes : ils permettent de représenter le fait qu'annuler une unité de flot sur un arc coûteux peut diminuer le coût total.

En revanche, un cycle négatif dans le graphe résiduel est problématique. Si un tel cycle est exploitable, il peut permettre de réduire indéfiniment le coût sans changer la quantité de flot envoyée de la source vers le puits. C'est pour cette raison que le projet contient une détection explicite des cycles négatifs.

4.4 Version avec Dijkstra et renormalisation

La consigne demande aussi une variante avec Dijkstra et renormalisation des coûts. L'idée théorique consiste à utiliser des potentiels $p(v)$ pour transformer les coûts :

$$c'_p(u, v) = c(u, v) + p(u) - p(v)$$

Si les potentiels sont bien choisis, les coûts réduits deviennent non négatifs, ce qui permet d'utiliser Dijkstra tout en conservant les plus courts chemins corrects dans le graphe original.

Dans l'état actuel du dépôt, le fichier `src/Dijkstra_min_cost_flow.py` expose la même interface que `min_cost_flow.py` et sert à comparer les résultats dans `src/test.py`. La logique interne reste très proche de la version Bellman-Ford. Le test vérifie donc surtout que les deux chemins d'exécution produisent le même flot, le même coût et le même graphe résiduel. Pour une version Dijkstra complète, il faudrait remplacer la recherche `shortest_path` par un Dijkstra sur coûts réduits et mettre à jour les potentiels après chaque augmentation.

4.5 Résultat sur l'exemple de la consigne

Le fichier `data/test3.txt` modélise le problème de flot de coût minimum donné dans la consigne. Une source artificielle est reliée au sommet 0, le puits artificiel reçoit le flot depuis les sommets 3 et 4, et un arc de retour du puits vers la source est présent dans les données.

Sur ce fichier, le programme renvoie :

flot maximum = 20

coût total = 150

Le résultat montre que les 20 unités demandées peuvent être envoyées, tout en tenant compte des coûts des arcs internes.

5. Détection de cycles négatifs

5.1 Pourquoi c'est nécessaire

Dans un problème de flot de coût minimum, les cycles négatifs sont un cas à surveiller. Un cycle négatif signifie que l'on peut parcourir un cycle et diminuer le coût total. Si ce cycle est utilisable dans le graphe résiduel, l'optimalité de la solution devient suspecte.

C'est pour cela que le fichier `src/detect_negative_cycle.py` implémente une détection par Bellman-Ford.

5.2 Algorithme utilisé

Le principe est le suivant :

1. initialiser la distance de la source à 0 et les autres distances à $+\infty$;
2. relâcher tous les arcs $n - 1$ fois ;
3. refaire un passage sur les arcs ;
4. si une distance peut encore être améliorée, alors un cycle négatif est détecté.

Ce choix est classique et cohérent avec le reste du projet, car Bellman-Ford accepte les coûts négatifs.

5.3 Normalisation

Le fichier contient aussi une fonction `normalize_graph`. Elle ajoute une constante à tous les coûts afin de rendre les poids non négatifs.

Cette normalisation est utile pour produire un graphe plus simple à manipuler ou à visualiser, mais elle ne doit pas être confondue avec la méthode des potentiels utilisée pour Dijkstra. Ajouter la même constante à tous les arcs change le coût des chemins en fonction de leur nombre d'arcs. Cela peut donc modifier le choix du plus court chemin.

C'est une solution simple pour éviter des poids négatifs dans certains tests, mais pas une preuve d'optimalité pour le min-cost flow.

6. Visualisation et tests

6.1 Export DOT

Le fichier `src/graph_to_dot.py` permet d'exporter un graphe au format DOT. C'est un outil pratique pendant le développement, parce qu'il permet de voir les capacités résiduelles et les coûts après l'exécution des algorithmes.

Les arcs de retour peuvent être affichés en bleu grâce au champ `retour`. Les étiquettes affichent la capacité et le coût, ce qui aide à comprendre comment le graphe évolue après chaque augmentation.

6.2 Tests effectués

Les principaux tests lancés sont les suivants :

Fichier	Algorithme	Résultat obtenu
<code>data/test2.txt</code>	flot maximum	flot = 60
<code>data/test3.txt</code>	flot coût minimum	flot = 20, coût = 150
<code>data/graph_classic.txt</code>	comparaison des deux versions	flot = 23, coût = 174

Le fichier `src/test.py` compare notamment les sorties des deux modules de flot de coût minimum : `min_cost_flow.py` et `Dijkstra_min_cost_flow.py`. Il vérifie que les deux résultats sont identiques et que les graphes résiduels finaux sont égaux.

7. Choix et limites

7.1 Choix principaux

Le premier choix important a été de construire une structure de graphe résiduel explicite. Cela rend les algorithmes plus proches de leur description théorique : augmenter un flot revient simplement à modifier les capacités des arcs directs et inverses.

Le deuxième choix a été d'utiliser Bellman-Ford pour les chemins de coût minimum. C'est moins rapide que Dijkstra, mais plus robuste dans un premier temps, parce que les arcs de retour introduisent naturellement des coûts négatifs.

Le troisième choix a été de séparer les responsabilités :

- la structure du graphe est isolée dans un fichier dédié ;
- les algorithmes de flot sont dans leurs propres modules ;
- la détection de cycles négatifs est indépendante ;
- la visualisation est séparée de la logique algorithmique.

Cette séparation rend le code plus facile à modifier. Par exemple, on peut remplacer la recherche de plus court chemin dans le flot de coût minimum sans toucher à la structure du graphe.

7.2 Limites actuelles

Le projet répond aux points essentiels du calcul de flot et du graphe résiduel, mais certaines améliorations restent possibles.

La coupe minimale n'est pas encore retournée directement par `max_flow.py`. Elle est déductible du graphe résiduel final, mais une fonction dédiée rendrait le résultat plus clair.

La version Dijkstra avec potentiels n'est pas encore pleinement séparée de la version Bellman-Ford. Le fichier existe et les tests comparent les sorties, mais une implémentation complète devrait introduire les potentiels, les coûts réduits et une file de priorité.

Enfin, le flot sur chaque arc original n'est pas stocké explicitement. Le choix de ne

garder que les capacités résiduelles simplifie les mises à jour, mais oblige à reconstruire le flot final si l'on veut l'afficher sous la forme “flot / capacité” pour chaque arc initial.

8. Conclusion

Ce projet m'a permis de mieux comprendre le rôle du graphe résiduel dans les problèmes de flot. Au départ, on peut voir Ford-Fulkerson comme une simple répétition de chemins entre une source et un puits. En pratique, toute la subtilité vient de la possibilité de corriger les décisions précédentes grâce aux arcs de retour.

La structure commune **Graph/Edge** rend cette idée visible dans le code : les algorithmes ne manipulent pas une formule abstraite, ils modifient directement les capacités disponibles. Le flot maximum utilise cette structure pour saturer progressivement le réseau, tandis que le flot de coût minimum ajoute la notion de coût et choisit les chemins les moins chers.

Les tests réalisés sur les exemples de la consigne donnent des résultats cohérents : un flot maximum de 60 sur le premier réseau et un flot de 20 pour un coût total de 150 sur l'exemple de coût minimum. Le projet pourrait encore être amélioré avec l'affichage direct de la coupe minimale et une vraie version Dijkstra avec potentiels, mais la base algorithmique et la structure de données du graphe résiduel sont en place.